Inventory & Truck-Stock Platform — Test Plan

A step-by-step manual test run covering environment setup, operator onboarding, account creation, authentication, every web page, all back-end routes, email notifications, lead capture, and security spot-checks.

API · Live on Railway | Web · Next.js | Mobile · Flutter | Desktop · Qt

API: inventory-system-api-production.up.railway.app
Generated: June 18, 2026 · **Version:** 1.0

Table of Contents

1. How to use this document

This is a **step-by-step manual test plan** for the Inventory & Truck-Stock platform. Work top to bottom: the early sections create the data (super-admin → activation code → company) that later sections depend on. Each test lists **what to do** and the **expected result**; tick the box when it passes.

Symbol	Meaning
`▢`	Check to mark the test as passed
`API`	The back-end base URL (live: `https://inventory-system-api-production.up.railway.app`)
`WEB`	The web front-end base URL (your Vercel/Railway/host URL, or `http://localhost:3001`)

> Throughout, replace `$API` and `$WEB` with your real URLs. Copy-paste the `curl` blocks into a terminal; they print the server's JSON so you can compare against "Expected".

---

2. Pre-flight: environment configuration

Before testing, confirm these are set. The API **refuses to start** in production if a required variable is missing or weak.

2.1 Back-end (Railway → Variables)

Variable	Required	Example / note
`DATABASE_URL`		PostgreSQL connection string
`JWT_SECRET`		32+ random chars (`openssl rand -hex 32`)
`JWT_REFRESH_SECRET`		Different 32+ random chars
`SUPER_ADMIN_BOOTSTRAP_SECRET`	—	Optional. Set it (16+ chars) to use the `/admin/setup` web bootstrap; leave it blank and use `npm run seed` instead
`CORS_ORIGINS`	(prod)	Must include your `WEB` origin, comma-separated
`NODE_ENV`	—	`production` on Railway
`RESEND_API_KEY` *or* `SMTP_*`	—	One email provider to actually send email (§9). Resend recommended on Railway
`ADMIN_NOTIFICATION_EMAIL`	—	Where sign-up/lead alerts go
`FRONTEND_URL`	—	Used in password-reset links

▢ **2.1.1** The three secrets you already set are present: `JWT_SECRET`, `JWT_REFRESH_SECRET`, `SUPER_ADMIN_BOOTSTRAP_SECRET`. ▢ **2.1.2** `CORS_ORIGINS` includes your web app's exact origin (scheme + host, no path).

2.2 Web (host build settings)

Variable	Required	Example
`NEXT_PUBLIC_API_BASE_URL`		`https://inventory-system-api-production.up.railway.app`
`NEXT_PUBLIC_SITE_URL`	—	Your public web URL

▢ **2.2.1** `NEXT_PUBLIC_API_BASE_URL` points at the live API (no trailing slash). ▢ **2.2.2** On the web **Settings** page, the shown "API connection" matches `$API`.

3. Smoke tests — API is alive

```
API=https://inventory-system-api-production.up.railway.app

curl -s $API/health          # liveness
curl -s $API/ready           # readiness (checks the database)
curl -s $API/                # root banner
```

☐ **3.1** `GET /health` → `{"status":"ok", ...}` (HTTP 200). ☐ **3.2** `GET /ready` → `{"status":"ready", ...}` (200). If `503`, the DB is unreachable. ☐ **3.3** `GET /` → `Inventory System API`. ☐ **3.4** `GET /openapi.json` → returns the OpenAPI document. ☐ **3.5** Unknown route, e.g. `curl -s $API/nope` → JSON `404` (not an HTML page).

4. Operator onboarding (super-admin)

This is the fix for "can't create a test account": you need an **activation code**, and an operator mints it. Pick **one** path (A is easiest).

4.1 Path A — Web console (recommended)

▢ **4.1.1** Open `$WEB/admin/setup`. Enter an email, password, and your `SUPER_ADMIN_BOOTSTRAP_SECRET`; submit. → You are redirected to `/admin`. ▢ **4.1.2** Re-open `$WEB/admin/setup` and submit again → it fails with "Super admin already exists" (the bootstrap endpoint is now closed). expected. ▢ **4.1.3** Sign out, then sign in at `$WEB/admin/login` with the same credentials → lands on `/admin`. ▢ **4.1.4** On `/admin`, in **Create an activation code**, keep the random code (e.g. `ABCD-1234-EFGH`), choose plan `PRO`, set limits, click **Create code**. → It appears in the **Activation codes** table marked **Available**. ▢ **4.1.5** Copy the code (clipboard button). Keep it for §5.

4.2 Path B — Seed script (one command)

Run in a Railway one-off shell (or locally with `DATABASE_URL` set):

```
npm run seed
```

▢ **4.2.1** Output shows `Super-admin created` and `Activation code created: TEST-2026`. ▢ **4.2.2** Re-run `npm run seed` → it reports both already exist (idempotent, no dupes).

4.3 Path C — Raw API

```
SEC=your-bootstrap-secret

# create operator
curl -s -X POST $API/super-admin/create -H "Content-Type: application/json" \
  -H "x-bootstrap-secret: $SEC" \
  -d '{"email":"root@example.com","password":"StrongPass123"}'

# login → copy the "token"
curl -s -X POST $API/super-admin/login -H "Content-Type: application/json" \
  -d '{"email":"root@example.com","password":"StrongPass123"}'

TOKEN=paste-token-here
curl -s -X POST $API/super-admin/activation-codes \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"code":"TEST-2026","plan":"PRO","maxUsers":25,"maxProducts":5000}'
```

▢ **4.3.1** `create` → `201` with the new super-admin (only works once). ▢ **4.3.2** `create` **without** the `x-bootstrap-secret` header → `403`. ▢ **4.3.3** `login` → `{ "token": "..." }`. ▢ **4.3.4** `activation-codes` with the token → `201` with the code. ▢ **4.3.5** `activation-codes` **without** the token → `401`.

5. Create a test account (registration)

Use the activation code from §4.

5.1 Web

▢ **5.1.1** Open `$WEB/register`. Enter Company name, work email, password (6+ chars), and the activation code. Submit. → Redirected to `/dashboard`, signed in. ▢ **5.1.2** In the operator console `/admin`, the new company now appears under **Companies**, and the activation code shows **Used**.

5.2 API (equivalent)

```
curl -s -X POST $API/auth/register -H "Content-Type: application/json" \
  -d '{"companyName":"Cascade
HVAC","email":"admin@cascade.test","password":"secret123","code":"TEST-2026"}'
```

▢ **5.2.1** → `200/201` with `accessToken`, `refreshToken`, and `user` (role `ADMIN`). ▢ **5.2.2** Re-using the **same** code → `401 Invalid activation code` (single-use). ▢ **5.2.3** A bogus code → `401 Invalid activation code`. ▢ **5.2.4** Missing a field → `400` with a validation message.

6. Authentication

```
# Login
curl -s -X POST $API/auth/login -H "Content-Type: application/json" \
  -d '{"email":"admin@cascade.test","password":"secret123"}'
```

▢ **6.1** Login with correct credentials → tokens + user. ▢ **6.2** Login with wrong password → `401 Invalid credentials`. ▢ **6.3** `GET /auth/validate` with `Authorization: Bearer <accessToken>` → `200`. ▢ **6.4** `GET /auth/validate` with no/garbage token → `401`. ▢ **6.5** `POST /auth/refresh` with `{"refreshToken":"..."}` → a fresh `accessToken`. ▢ **6.6** `POST /auth/request-reset` `{"email":"admin@cascade.test"}` → generic `200` ("If this email exists…") — never reveals whether the email exists. ▢ **6.7** Web: **Settings** → **Change password** with the right current password → success, and you can log in with the new password afterwards. ▢ **6.8** `POST /auth/logout` with the refresh token → `200`; reusing that refresh token on `/auth/refresh` afterwards → `401` (it was revoked).

7. Web app — page by page

Sign in as the company admin (§5), then walk every page.

▢ **7.1 Login/redirect** Visiting `$WEB/dashboard` while signed out redirects to `/login`. ▢ **7.2 Dashboard** Loads stat cards (products, low-stock, assets) without errors. ▢ **7.3 Products** List renders. **Create** a product (name + barcode) → appears in list. ▢ **7.4 Products — scan** Scan-in increases quantity; scan-out decreases it. ▢ **7.5 Products — low stock** A product below its threshold is flagged. ▢ **7.6 Products — edit/delete** Editing persists; deleting removes it. ▢ **7.7 Assets** Create an asset (name, type, serial) → appears; delete works. ▢ **7.8 Trucks** Truck-stock page loads trucks/templates/receipts (may be empty — OK). ▢ **7.9 Reports** Inventory and assets summaries render numbers/charts. ▢ **7.10 Reports — export** Export buttons download CSV/XLSX/PDF files that open cleanly. ▢ **7.11 Settings** Profile + API connection show correctly; sign-out works. ▢ **7.12 Marketing** `/`, `/features`, `/pricing` render; nav and footer links work. ▢ **7.13 404** A nonsense path (e.g. `$WEB/nope`) shows the styled not-found page.

8. Backend route reference (authenticated)

Set `T` to a company-admin access token: `T=<accessToken from §6>`. All calls send `-H "Authorization: Bearer $T"`.

#	Method & path	Purpose	Expected
8.1	`GET /products`	List products	`200` array
8.2	`POST /products`	Create `{name,barcode}`	`201` product
8.3	`PUT /products/:id`	Update a product	`200`
8.4	`DELETE /products/:id`	Delete a product	`200/204`
8.5	`GET /products/low-stock`	Below threshold	`200` array
8.6	`POST /products/scan-in`	`{barcode,quantity}`	`200`, qty up
8.7	`POST /products/scan-out`	`{barcode,quantity}`	`200`, qty down
8.8	`GET /assets`	List assets	`200` array
8.9	`POST /assets`	Create `{name,type,serialCode}`	`201`
8.10	`DELETE /assets/:id`	Delete an asset	`200/204`
8.11	`GET /reports/inventory-summary`	Inventory totals	`200` object
8.12	`GET /reports/assets-summary`	Asset totals	`200` object
8.13	`GET /exports/products/csv`	CSV export	file download
8.14	`GET /exports/products/xlsx`	Excel export	file download
8.15	`GET /exports/products/pdf`	PDF export	file download
8.16	`GET /truck-stock/trucks`	Trucks	`200` array
8.17	`GET /truck-stock/templates`	Templates	`200` array
8.18	`GET /users`	Company users	`200` array

▢ **8.A** Each row returns its expected status with a valid token. ▢ **8.B** Any of the above **without** a token → `401`. ▢ **8.C** A product id from **another** company → `404` (tenant isolation; never another tenant's data).

9. Email & notifications

Requires an email provider configured (§2.1): `RESEND_API_KEY` (recommended on Railway, as Railway blocks outbound SMTP) **or** the `SMTP_*` vars. Inbox to watch: `ADMIN_NOTIFICATION_EMAIL` (`felipetiburcioviana@gmail.com`).

> Submissions never hang on email — the API responds immediately and sends in the background. If an email doesn't arrive, check the API logs for `Email delivery failed` (Resend/SMTP) or `Email not sent: no provider configured`.

▢ **9.1** Registering a company (§5) → a **"New company registered"** email arrives at the admin inbox, and a **welcome** email arrives at the new user's address. ▢ **9.2** Creating the first super-admin (§4) → a **"Super-admin account created"** email. ▢ **9.3** Submitting the demo form (§10) → a **"New demo / contact request"** email with the submitted fields. ▢ **9.4** `POST /auth/request-reset` for a real user → a **password reset** email with a working link (opens `$FRONTEND_URL/reset-password?token=…`). ▢ **9.5** With SMTP **unset**: the same actions still succeed (HTTP 200) and the API logs "Email not sent: SMTP is not configured" — no crashes. (graceful degradation)

> Gmail: use an **App Password** (2-Step Verification required), not the account password.

---

10. Lead capture (request a demo)

▢ **10.1** Web: open `$WEB/request-demo`, fill first name, work email, company; submit → success state ("you're on the list"). ▢ **10.2** API direct:

```
curl -s -X POST $API/leads -H "Content-Type: application/json" \
  -d '{"firstName":"Pat","email":"pat@acme.test","company":"Acme"}'
```

→ `201` with a thank-you message. ▢ **10.3** Missing email or company → `400` validation error. ▢ **10.4** Submit the form 9+ times quickly → after 8/hour you get `429 Too many requests` (anti-spam rate limit).

---

11. Security & resilience spot-checks

▢ **11.1 Auth brute-force** 11 rapid `POST /auth/login` attempts → `429` after 10. ▢ **11.2 Super-admin protection** `POST /super-admin/activation-codes` without a token → `401`. ▢ **11.3 CORS** From a browser on a non-allowlisted origin, an API call is blocked; from your real `WEB` origin it succeeds. (If the web app shows network/CORS errors, add its origin to `CORS_ORIGINS`.) ▢ **11.4 Security headers** `curl -sI $WEB` shows `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`. ▢ **11.5 Forced password change** A user with `mustChangePassword` is routed to change it before using the app.

12. Mobile app (Flutter) — high level

Requires a build on a device/emulator (`InventoryQtApp/mobile`).

▢ **12.1** App launches without crashing; login screen appears. ▢ **12.2** In **Settings**, the API base URL defaults to the production HTTPS URL and can be overridden. ▢ **12.3** Log in as a technician/admin → dashboard loads. ▢ **12.4** Barcode **scan-in/scan-out** updates stock. ▢ **12.5 Receipt** photo upload succeeds. ▢ **12.6** Token auto-refresh: after the access token expires, the app keeps working without a forced re-login.

---

13. Desktop app (Qt) — high level

Requires a Windows build (`InventoryQtApp/InventoryQtApp`).

▢ **13.1** App starts; login window appears; version shows in the title/about. ▢ **13.2** The API URL resolves to production (env → QSettings → built-in default). ▢ **13.3** Log in → dashboard/pages load. ▢ **13.4** Product and asset CRUD works. ▢ **13.5** Excel export produces a valid file. ▢ **13.6** A rolling log file is written under `%APPDATA%/InventorySystem/` .

14. Sign-off summary

Area	Result (Pass / Fail / N/A)	Notes
3. API smoke		
4. Operator onboarding		
5. Registration (test account)		
6. Authentication		
7. Web pages		
8. Backend routes		
9. Email & notifications		
10. Lead capture		
11. Security spot-checks		
12. Mobile		
13. Desktop		

Tester: ___ Date: __ Build/commit: _____